REPORT TECNICO PENETRATION TESTING

Esecuzione Esercitazione Pratica: XSS Reflected & UNION-based SQL Injection

Studente: Simone Meola

Piattaforma Training: Epicode Cybersecurity Lab

Ambiente Virtualizzato: VirtualBox (Host-Only Network)

Macchine Coinvolte: Kali Linux (Attacker) vs Metasploitable (Target)

1. SINTESI ESECUTIVA & SCENARIO OPERATIVO

Il presente documento illustra le attività svolte da **Simone Meola** durante il laboratorio di sicurezza informatica focalizzato sull'audit e sullo sfruttamento pratico di due fondamentali vulnerabilità applicative web presenti nella Damn Vulnerable Web Application (DVWA): **Cross-Site Scripting (XSS Reflected)** e **SQL Injection (UNION-base**

2. CONFIGURAZIONE DELLA RETE E LIVELLO DI SICUREZZA

Come primo step metodologico, è stata verificata la connettività di rete ICMP bidirezionale tra la macchina attaccante Kali Linux (IP `192.168.56.101`) e la macchina target Metasploitable (IP `192.168.56.102`), ed è stato impostato il livello di sicurezza applicativo su **Low**.

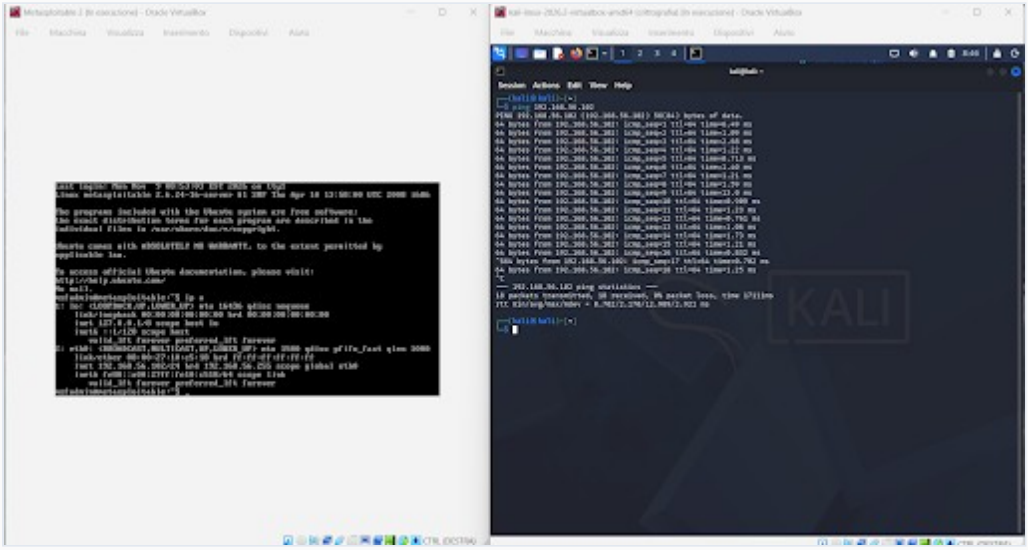


Figura 1: Esito positivo del comando 'ping 192.168.56.102' da terminale Kali verso il target (0% packet loss).

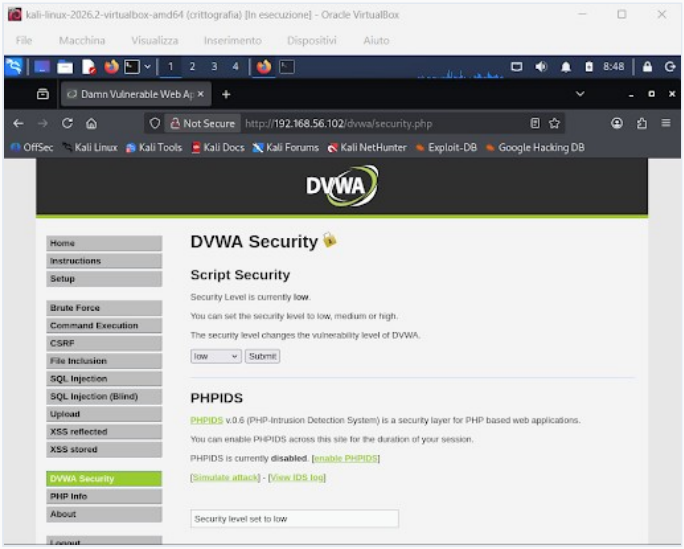


Figura 2: Impostazione del parametro DVWA Security su livello 'Low' per la disattivazione dei controlli di sanitizzazione.

3. SFRUTTAMENTO VULNERABILITÀ REFLECTED XSS

Nel modulo XSS Reflected, l'applicazione riceve l'input dell'utente e lo riflette direttamente nel DOM della pagina senza codifica dei caratteri speciali. È stato inserito il seguente script di test:

```
<script>alert('XSS Test by me');</script>
```

Esecuzione Codice Arbitrario (XSS)

Inserendo il tag `<script>` nel campo "What's your name?", il browser del client ha interpretato il codice ed eseguito la funzione JavaScript `alert()`, confermando la piena vulnerabilità al Cross-Site Scripting.

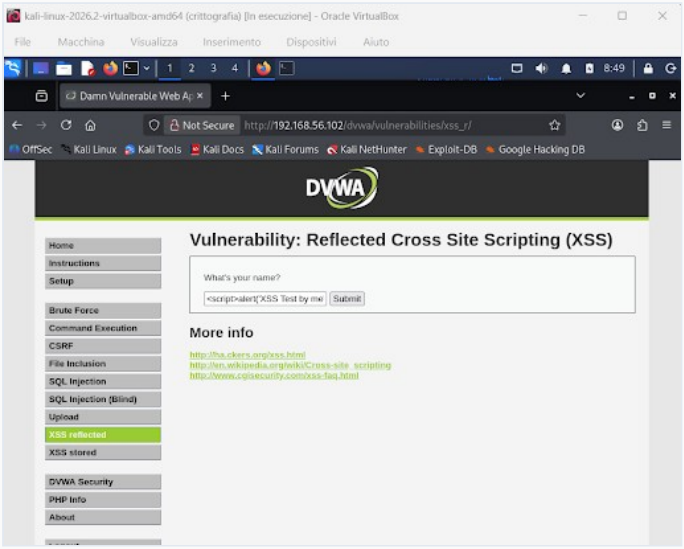


Figura 3: Inserimento del payload JavaScript nel form di input della sezione Reflected XSS.

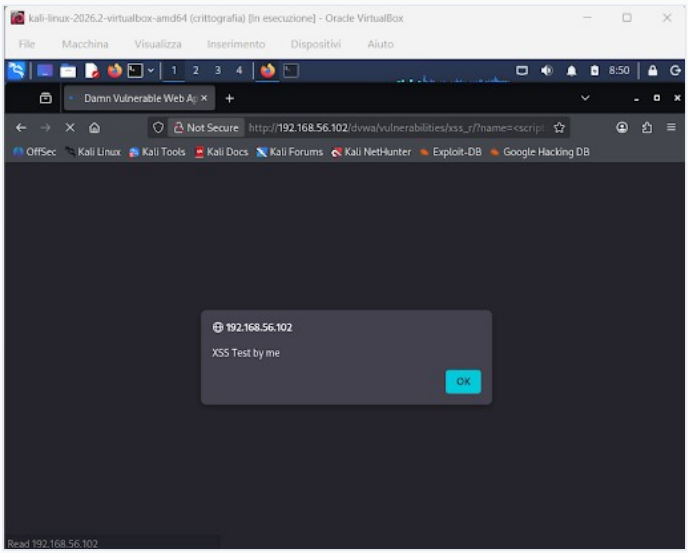


Figura 4: Esecuzione confermata del pop-up modale 'XSS Test by me' nel browser.

4. SFRUTTAMENTO SQL INJECTION (UNION-BASED)

L'attacco SQL Injection sfrutta l'assenza di sanitizzazione nel parametro `id` HTTP. Il primo obiettivo consiste nel determinare il numero di colonne gestite dalla query SQL originale tramite l'operatore `UNION`.

Fase 4.1: Determinazione delle Colonne (Test NULL)

È stato inviato il seguente payload di test per forzare l'unione con 2 colonne nulle:

```
' UNION select null, null -- -
```

Analisi del payload: L'apice singolo `'` chiude la stringa originale, `UNION select null, null` tenta una query a 2 colonne, mentre `-- -` commenta la sintassi successiva. L'assenza di errori conferma che la tabella contiene esattamente **2 colonne**.

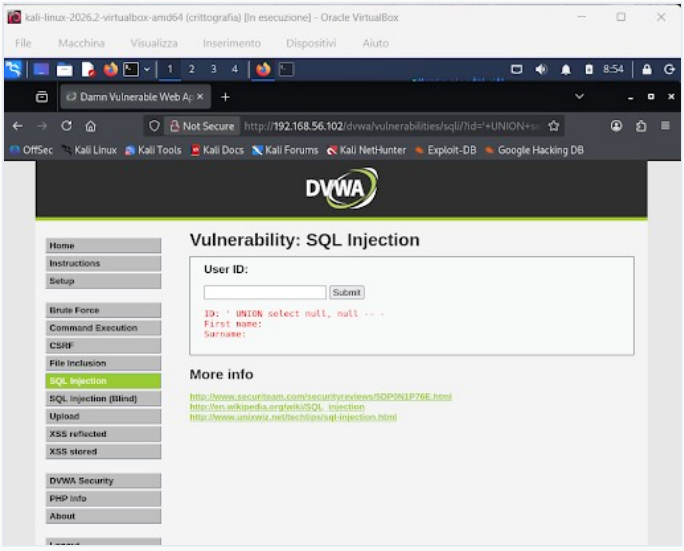


Figura 5: Risposta positiva dell'applicazione al payload 'UNION select null, null -- -' (Nessun errore di sintassi).

Fase 4.2: Mapping dello Schema del Database

Con la conferma delle 2 colonne, è stata interrogata la tabella di sistema `information_schema.columns` per estrarre l'elenco delle tabelle e delle relative colonne appartenenti al database attivo:

```
' UNION select table_name, column_name FROM information_schema.columns WHERE table_schema=database() -- -
```

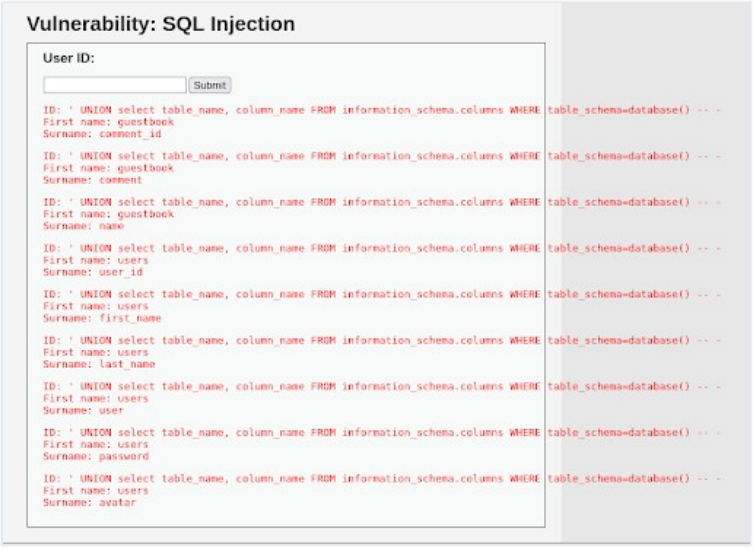


Figura 6: Estratto dello schema del database: individuata la tabella 'users' e le colonne 'user' e 'password'.

Fase 4.3: Esfiltrazione delle Credenziali (Exploit Finale)

Individuata la tabella target `users` e le colonne rilevanti `user` e `password`, è stato eseguito il comando finale per l'estrazione completa degli account memorizzati:

```
' UNION select user, password FROM users -- -
```



Figura 7: Risultato dell'esfiltrazione: dump completo di nomi utente e relativi hash MD5 delle password.

5. TABELLA DELLE CREDENZIALI ESFILTRATE

Account Username	Hash Password Estratto (MD5)	Password in Chiaro (Decifrata)	Stato Exploit
admin	`5f4dcc3b5aa765d61d8327deb882cf99`	password	**Compromesso**
gordonb	`e99a18c428cb38d5f260853678922e03`	abc123	**Compromesso**
1337	`8d3533d75ae2c3966d7e0d4fcc69216b`	charlie	**Compromesso**
pablo	`0d107d09f5bbe40cade3de5c71e9e9b7`	letmein	**Compromesso**
smithy	`5f4dcc3b5aa765d61d8327deb882cf99`	password	**Compromesso**

6. MATRICE DELLE VULNERABILITÀ E RECOMMENDATIONS

- **Mitigazione SQL Injection:** Sostituire le query dinamiche con *Prepared Statements* e query parametrizzate (PDO MySQLi) per separare nettamente il codice dai dati.
- **Mitigazione XSS:** Applicare la codifica contestuale dell'output (es. `htmlspecialchars()` in PHP) ed implementare politiche stringenti di *Content Security Policy (CSP)*.